

CSC1029 OO Programming

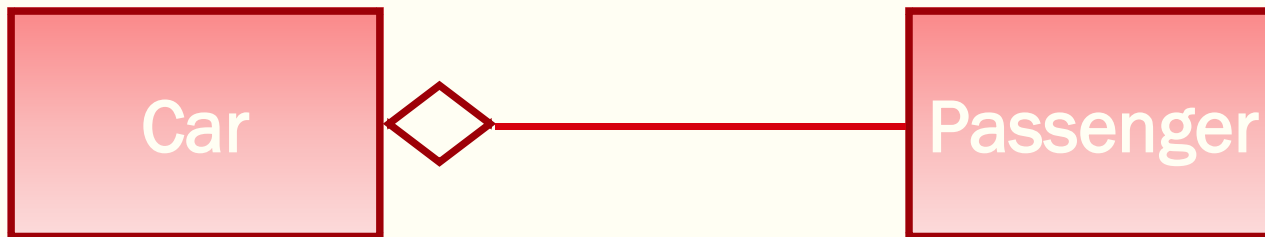
Lecture 16 – An Overview of Inheritance



Recap on the Previous lecture (1)



The diagram represents an **association** between the *Student* and *Module* classes.

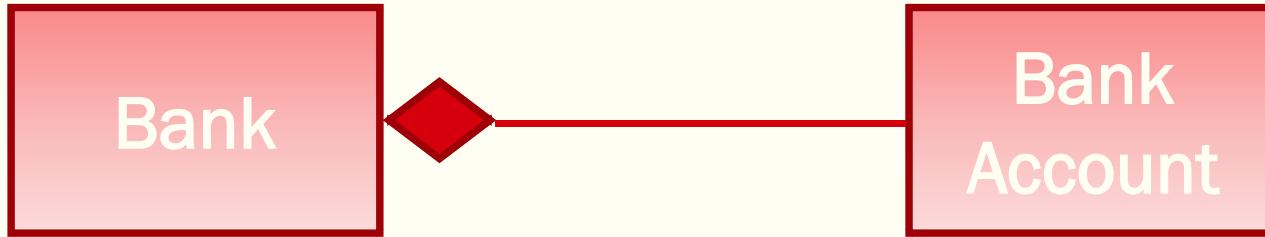


Aggregation is a more specialised type of association – and can be described in simple terms as "an object of one class can own or access the objects of another class."

A car can have many passengers – they come and go.

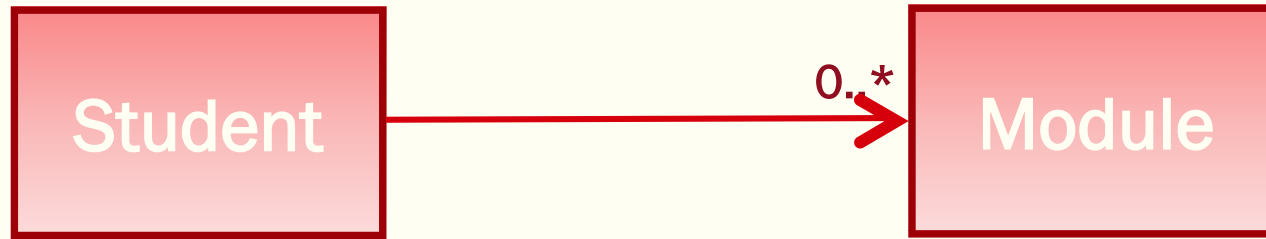
Delete a car object – the passengers continue to exist.

Recap on the Previous lecture (2)



Composition again, is a more specialised type of association – simply put, a subordinate object in the relationship ***will only exist*** for the lifetime of the containing or composite object.

Recap on the Previous lecture (3)



-Implementation of the relationship between *Student* and *Module*

-Development of the *Student* class:

- use of *ArrayList<Module>*
- implementation of *construction*
- implementation of *addModule()*
- implementation of *getModules()*
- implementation of *display()*

Recap on the Previous lecture (4)



Redevelopment of existing *Student* class:

- use of *ArrayList<Module>*
- changes to *construction*
- changes to *addModule()*
- changes to *getModules()*
- changes to *display()*

```
public class Student {  
    private int sNumber;  
    private String fName;  
    private String lName;  
    //private String modules;  
    private ArrayList<Module> mods;  
    ...  
}
```

Old (String) representation for modules replaced with a parameterised *ArrayList*

Recap on the Previous lecture (5)



Redevelopment of existing *Student* class:

- use of *ArrayList<Module>*
- changes to *construction*
- changes to *addModule()*
- changes to *getModules()*
- changes to *display()*

```
public Student(...) {
    this.sNumber = number;
    this.fName = first;
    this.lName = last;
    //modules = "";
    mods = new ArrayList<Module>();
}
```

Creation of an *ArrayList* ...

Recap on the Previous lecture (6)



Redevelopment of existing *Student* class:

- use of *ArrayList<Module>*
- changes to *construction*
- changes to *addModule()*
- changes to *getModules()*
- changes to *display()*

```
public boolean addModule(Module mod) {
    if (mod != null &&
        mods.size() < MAX_MODULES) {
        mods.add(mod);
        return true;
    }
    return false;
}
```

... add to the *ArrayList* – after validation

Recap on the Previous lecture (7)



Redevelopment of existing *Student* class:

- use of *ArrayList<Module>*
- changes to *construction*
- changes to *addModule()*
- changes to *getModules()*
- changes to *display()*

```
public String getModules() {
    String list = "";
    for(Module aMod : mods) {
        list += aMod.getCode() +
            " " +
            aMod.getName() + "\n";
    }
    return list;
}
```

... iterate through the *ArrayList* of Modules
... add the details of each module to a String

Recap on the Previous lecture (8)



Redevelopment of existing *Student* class:

- use of *ArrayList<Module>*
- changes to *construction*
- changes to *addModule()*
- changes to *getModules()*
- changes to *display()*

```
public String display() {
    String result = "";
    result += "Number: " +
        getNumber() + "\n";
    result += "Name: " +
        getName() + "\n";
    result += "Modules: ";
    if (mods.size() > 0) {
        result += getModules() + "\n";
    }
    else {
        result += "No modules.";
    }
    return result;
}
```

... add details of the Student and associated Modules to a String

Recap on the Previous lecture (9)



Implementing *StudentTester* ...

```
Student stu1 = new Student(12345678, "Homer", "Simpson");
Module prog = new Module("CSC1027",
                        "Programming", "The basics of OOP", 20, 36);
Module arch = new Module("CSC1036",
                        "Architecture", "All about hardware", 20, 36);

stu1.addModule(prog);
stu1.addModule(arch);
System.out.println(stu1.display());
```

*Student.java, Module.java &
TestStudent.java*



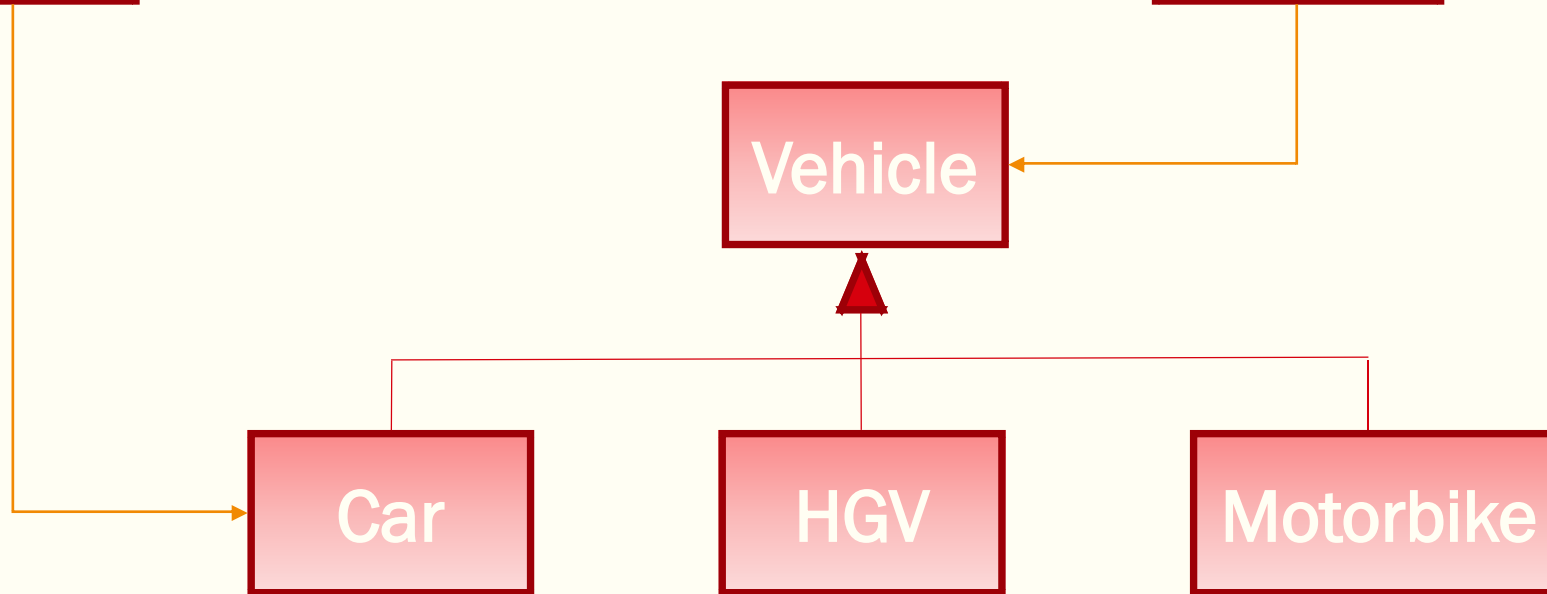
What's to come in today's lecture?

- Defining Inheritance
- Inheritance Hierarchies
- Java and Inheritance

What is Inheritance? (1)

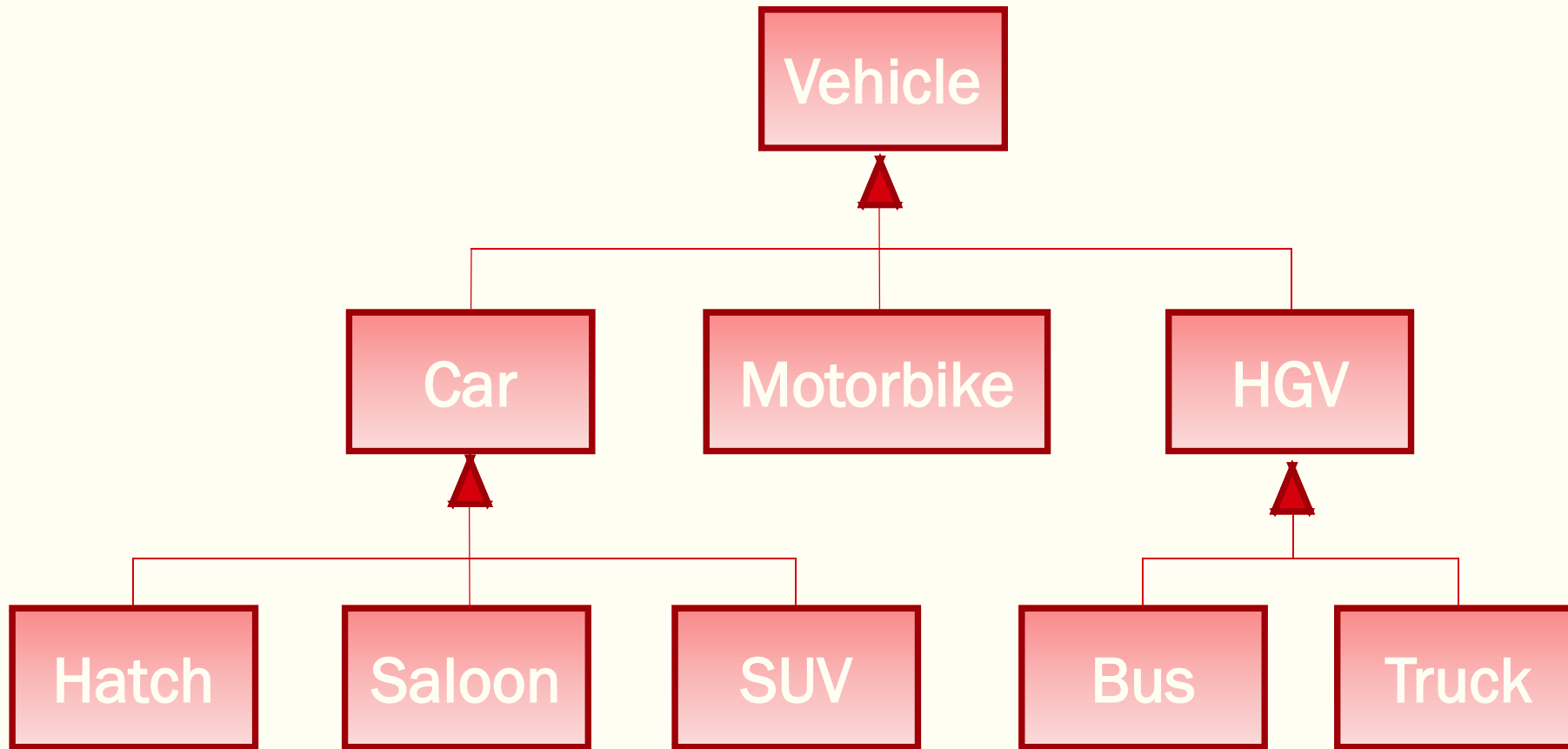
Following on from our definition of class *relationships* (associations) - In object-oriented design, *inheritance is a relationship* between a *more general class* (called the *superclass*) and a *more specialized class* (called the *subclass*).

The **subclass** inherits data and behaviour from the **superclass**.



What is Inheritance? (2)

This concept can be extended further ...



What is Inheritance? (3)

Inheritance is the concept that allows us to define *general* object classes

- These will have *states* and *behaviours*

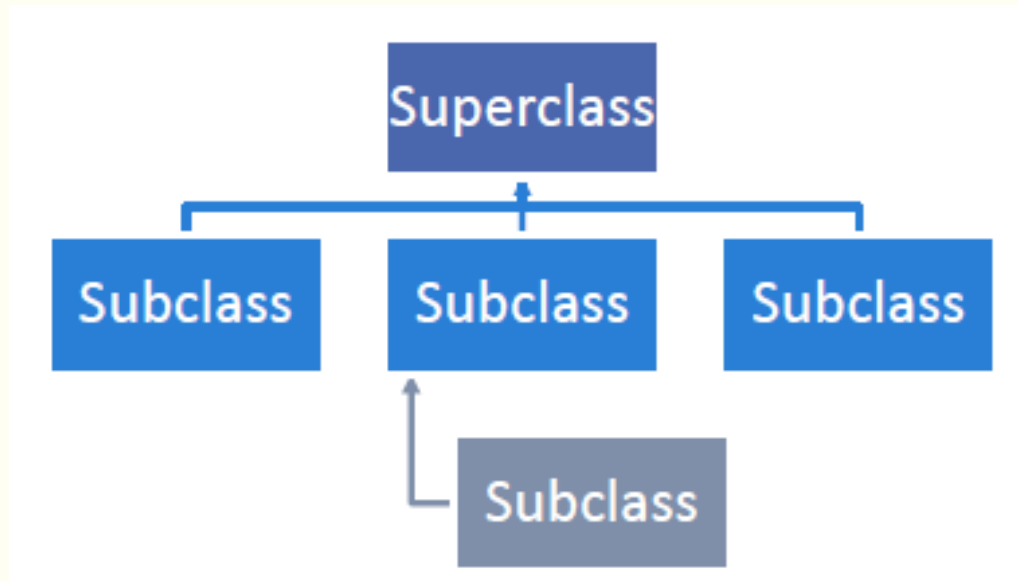
And then create further object *subclasses* based on or *inheriting* from the *general* classes

- Which can also have their own states and behaviours

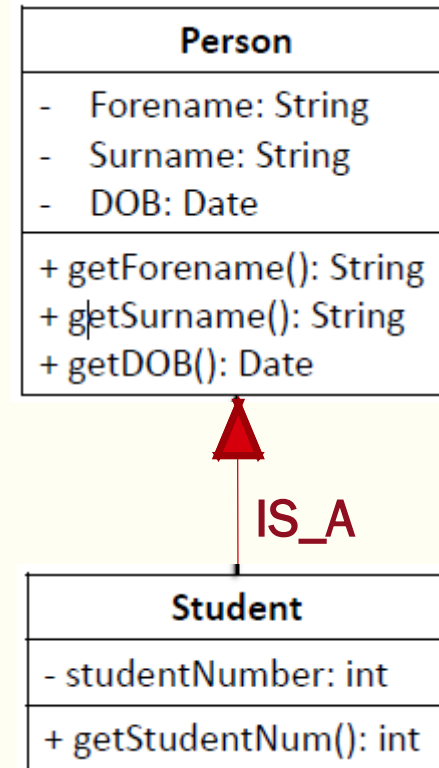
The subclasses *maintain the same behaviour as the parent class* but then can have individual behaviours as well

What is Inheritance? (4)

Inheritance relationships are defined in **UML** ... through the **IS_A** relationship



A Student **IS_A** Person



Inheritance – What's the Point?

- To reuse code we have already implemented
- To allow for customisation
- To provide easy maintenance
- To model the real world
- Ultimately to save time!

Inheritance – The IS_A Relationship

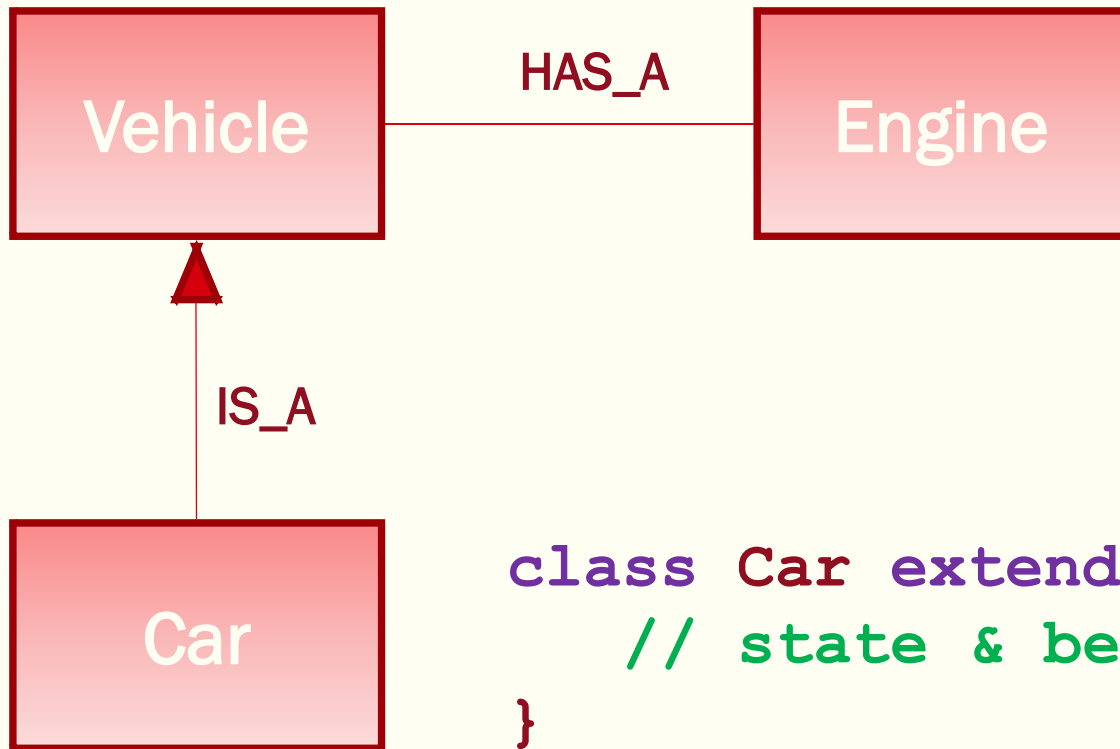
When we think about inheritance we should use the IS_A relationship

- Car IS_A Vehicle?
- Student IS_A Person?

When you are designing your code and want to know if an object inherits from another ask the question

Does it make sense that Y IS_A Z?

IS_A vs HAS_A



```
class Vehicle {  
    // state & behaviour  
    Engine eng;  
}
```

```
class Car extends Vehicle {  
    // state & behaviour  
}
```

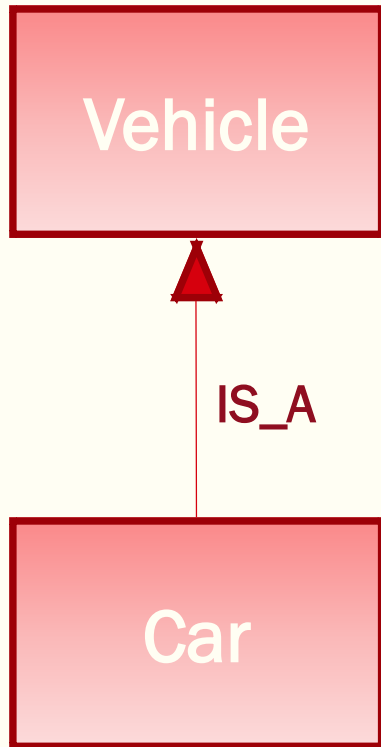
What type of relationship between Vehicle & Engine? **HAS_A** represents *aggregation* in this case.

Inheritance in Java (1)

```
class SuperclassName{  
    //states and behaviours  
}
```

```
class SubclassName extends SuperclassName{  
    //states and behaviours  
}
```

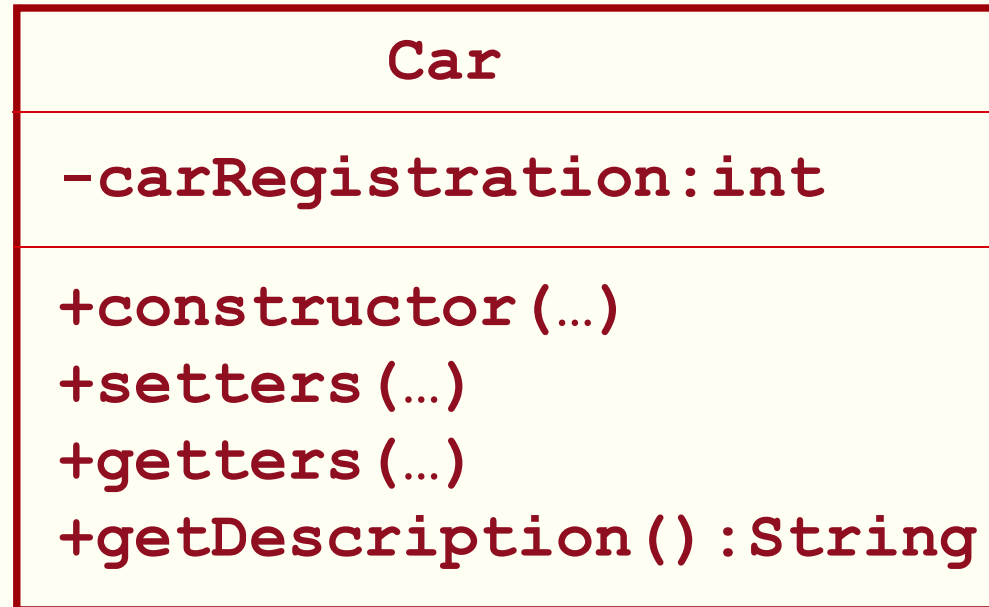
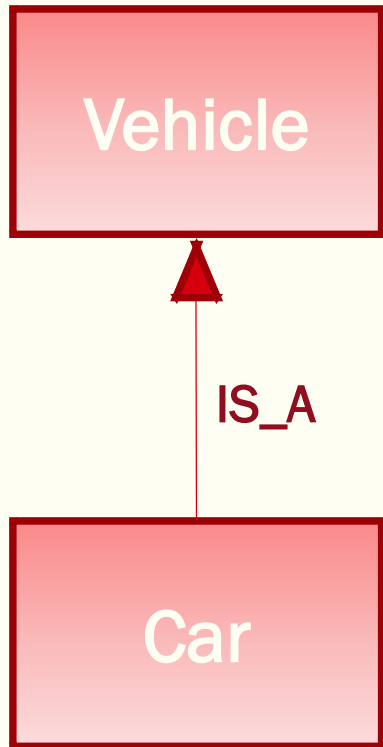
Inheritance in Java (2)



Vehicle
-numberOfTyres:int -engineSize:double -colour:String
+constructor (...) +setters (...) +getters (...) +printDetails ():String

Vehicle.java

Inheritance in Java (3)



Car.java

CarTester.java

Inheritance in Java (4)

Inheritance is a **one way** relationship ...

- A child inherits from a parent
- NOT the other way round

If you instantiate a variable of type Vehicle, it **does not** have access to the Car class states and behaviours

If you instantiate a variable of type Car, it does have access to the Vehicle class states and behaviours

Subclasses are more specific than the superclass they extend

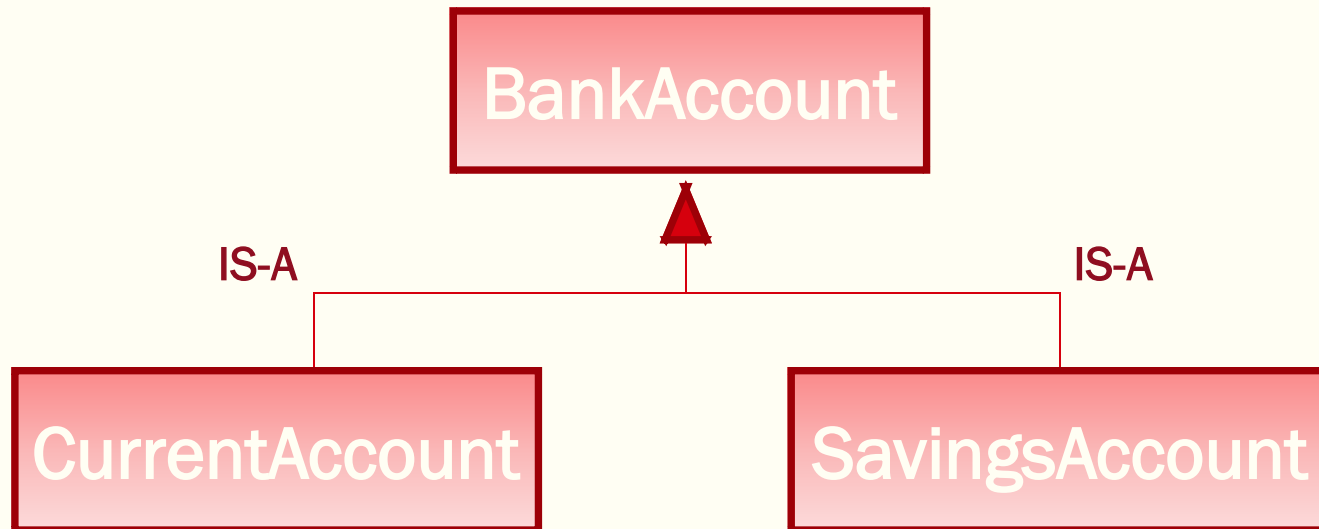
Inheritance in Java (5)

There is an **instanceOf** operator in Java which tells us if an instance of an object is a member of a particular class

```
Car aLimo = new Car(8, 3.0, "White", "XXI 1234");  
System.out.println("aPlainCar is an instance of Vehicle: "  
+ (aPlainCar instanceof Vehicle));
```

See: *CarInstanceOfTester.java*

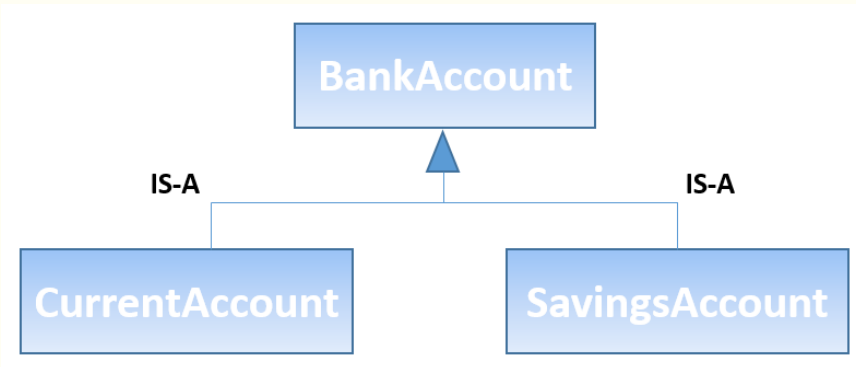
Inheritance in Java – BankAccount (1)



A Bank Account Hierarchy

BankAccount
- name : String - acctNum : int - balance : double - <u>nextNum : int = 1</u>
+BankAccount(String) +deposit(double):boolean +withdraw(double):boolean +statement():String -<u>nextAcctNum():int</u>

Inheritance in Java – BankAccount (2)



BankAccount
- name : String - acctNum : int - balance : double - <u>nextNum : int = 1</u>
+BankAccount(String) +deposit(double):boolean +withdraw(double):boolean +statement():String - <u>nextAcctNum():int</u>

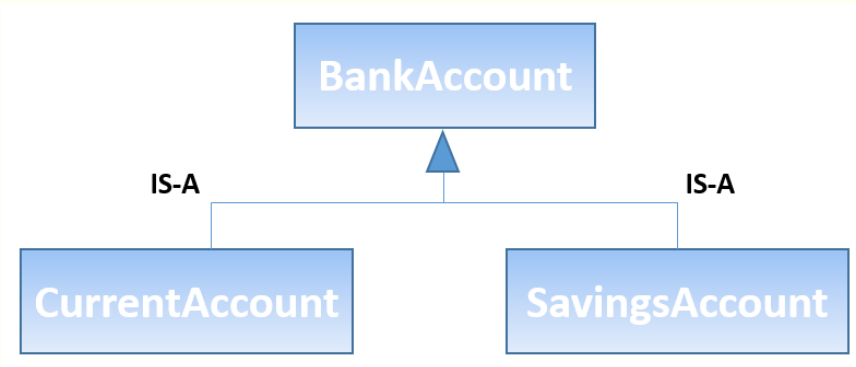
For any proposed **subclass** of BankAccount – what is inherited/visible?

Private methods/data of the **superclass** are **not visible** within a **subclass**.

Public methods/data of the **superclass** are **visible** within a **subclass**:

constructor
deposit()
withdraw()
statement()

Inheritance in Java – BankAccount (3)

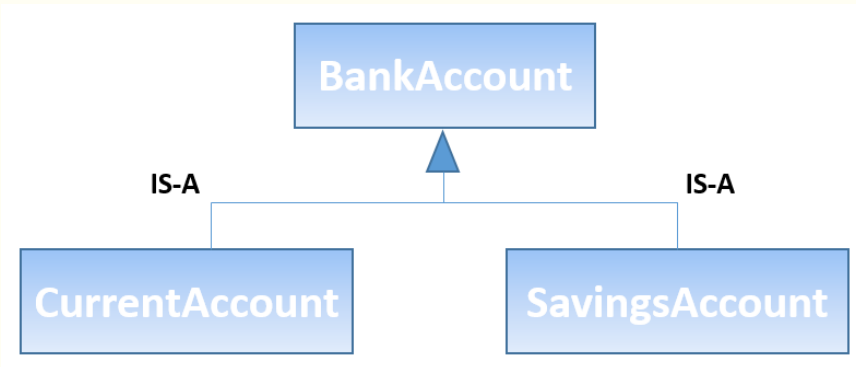


What specific attributes/behaviour could be added to subclass *SavingsAccount*?

BankAccount
- name : String - acctNum : int - balance : double - <u>nextNum : int = 1</u>
+BankAccount(String) +deposit(double):boolean +withdraw(double):boolean +statement():String -<u>nextAcctNum():int</u>

SavingsAccount
- interestRate : double
+ SavingsAccount(String) + getInterestRate(): double + setInterestRate(double):void + payInterest():void

Inheritance in Java – BankAccount (4)



BankAccount
- name : String - acctNum : int - balance : double - <u>nextNum : int = 1</u>
+BankAccount(String) +deposit(double):boolean +withdraw(double):boolean +statement():String - <u>nextAcctNum():int</u>

What specific attributes/behaviour could be added to subclass *CurrentAccount*?

CurrentAccount
- overdraftLimit : double
+ CurrentAccount(String) + getOverdraftLimit(): double + setOverdraftLimit(double):void

Potential problem – *withdraw()* from BankAccount does not allow a balance below £0.



What's Next?

More inheritance examples

Abstract classes

Interfaces

